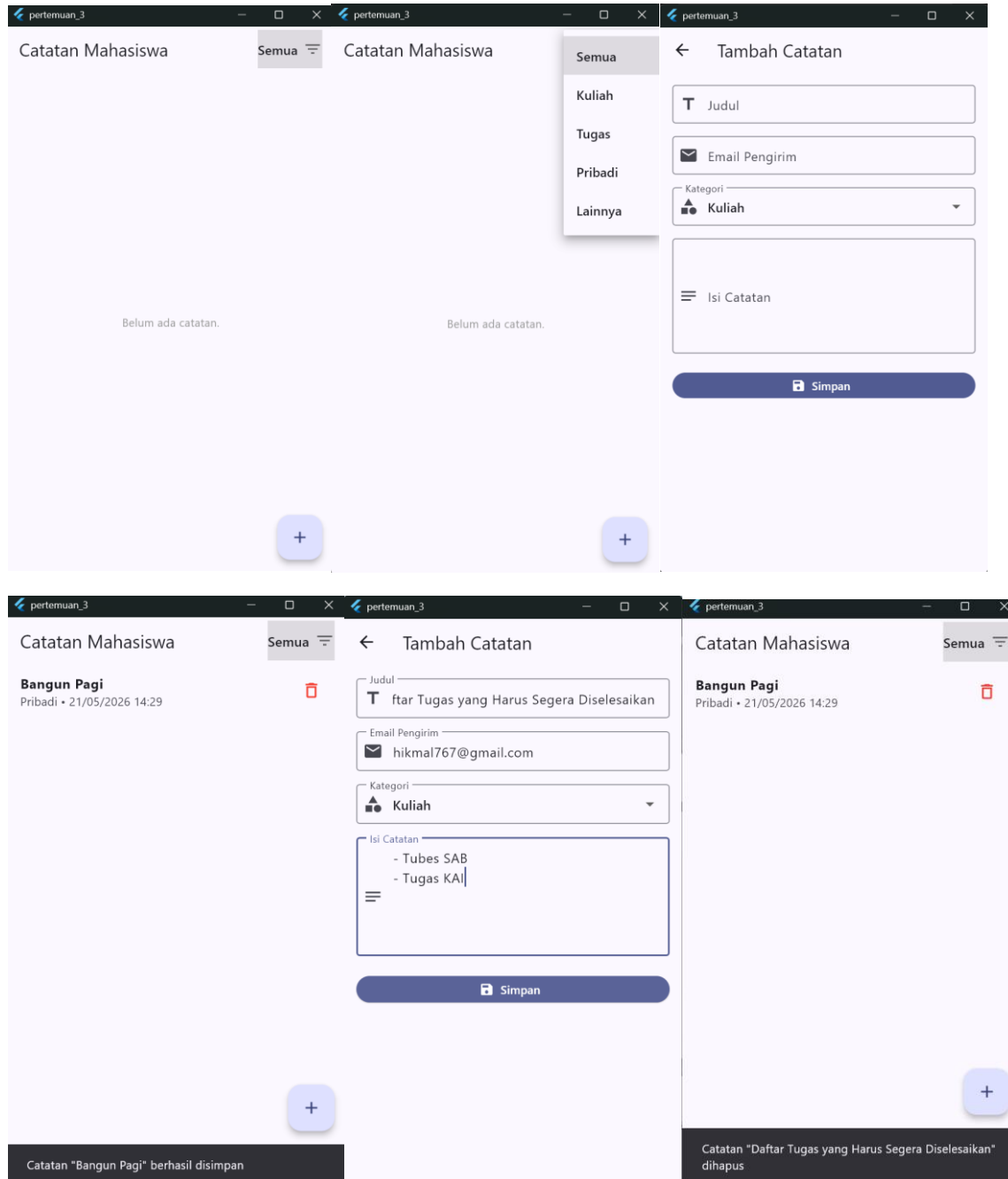


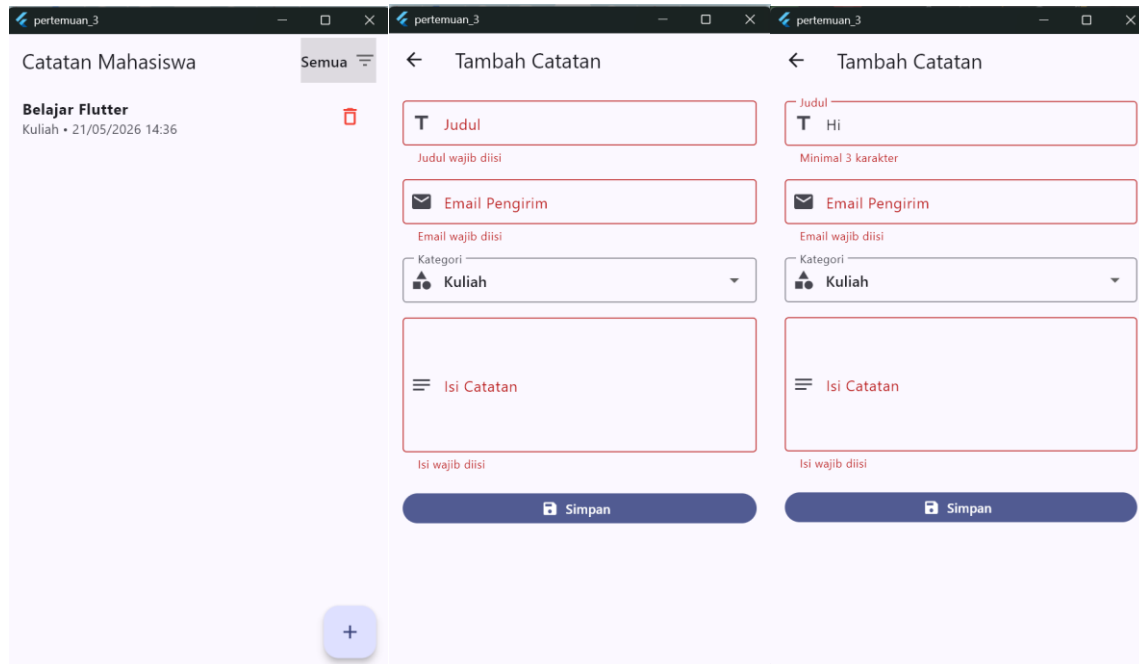
Nama: Hikmal Maulana

NPM: 233040091

Kelas: C

Hasil Screenshoot





Jawaban Pertanyaan Refleksi

1. Apa beda StatelessWidget dan StatefulWidget? Sebutkan satu contoh konkret kapan masing-masing dipakai.

 - a. StatelessWidget:**
Widget yang statis dan tidak memiliki *state* (data internal) yang berubah setelah pertama kali dibangun. Hanya dirender ulang jika dipicu oleh *parent*-nya.
Contoh: Menampilkan ikon, teks label sederhana, atau kartu profil statis yang datanya di-*hardcode*.
 - b. StatefulWidget**
Widget yang dinamis dan memiliki *state* internal. Ia bisa membangun ulang dirinya sendiri (*rebuild*) ketika datanya berubah.
Contoh: Halaman daftar yang isinya bisa bertambah/berkurang, form input teks, atau tombol counter angka.
2. Mengapa kita harus memanggil `setState()` saat mengubah data? Apa yang terjadi kalau lupa?

`setState()` berfungsi sebagai "pemberitahu" kepada framework Flutter bahwa ada data yang berubah, sehingga Flutter tahu ia harus memanggil fungsi `build()` ulang untuk merender tampilan terbaru di layar. Jika lupa memanggil `setState()`, datanya secara logika (di memori) akan berubah, tetapi UI atau tampilan di layar tidak akan ter-update (kelihatan *stale* atau diam saja).
3. Apa fungsi `GlobalKey<FormState>`? Mengapa tidak cukup `TextEditingController` saja?

GlobalKey<FormState> berfungsi sebagai "remot kontrol" untuk keseluruhan Form. Kunci ini digunakan untuk memicu proses validasi massal (validate()) pada semua TextFormField sekaligus, atau menyimpan seluruh data form (save()).

TextEditingController saja tidak cukup karena ia hanya bertugas membaca atau memanipulasi teks dari satu field secara individu. Ia tidak memiliki kemampuan mengevaluasi logika validasi grup (apakah form itu valid atau ada error).

4. Mengapa TextEditingController perlu di-dispose()?

TextEditingController mengikat resource atau memori di sistem (native resources) secara terus-menerus dan biasanya memiliki listener yang aktif berjalan. Jika tidak di-dispose() saat widget dihancurkan (misalnya saat pengguna pindah halaman), resource tersebut akan terus menumpuk di latar belakang dan menyebabkan kebocoran memori (memory leak), yang pada akhirnya membuat aplikasi lambat atau crash.

5. Apa beda Navigator.push dengan Navigator.pushNamed? Kapan pakai yang mana?

Navigator.push Melakukan navigasi secara langsung dengan membuat *route* baru menggunakan MaterialPageRoute di tempat (*on the fly*). Dipakai untuk aplikasi skala kecil, purwarupa (*prototyping*), atau jika *route*-nya sangat jarang dipanggil.

Navigator.pushNamed: Melakukan navigasi menggunakan *string* (misal: '/tambah') yang sudah didaftarkan sebelumnya di routes atau onGenerateRoute pada MaterialApp. Dipakai untuk aplikasi skala menengah ke atas karena jauh lebih rapi, terpusat, dan mudah dimaintenance.

6. Bagaimana cara mengirim data balik dari halaman B ke halaman A?

Data dikirimkan sebagai argumen kedua pada fungsi Navigator.pop() di halaman B. Contohnya: Navigator.pop(context, dataBaru); Kemudian, di halaman A, pemanggilan navigasinya harus bersifat *asynchronous* (await) untuk menangkap data tersebut. Contohnya: final hasil = await Navigator.pushNamed(...);

7. Mengapa kita perlu cek if (!mounted) return; setelah await?

Proses await (seperti menunggu pengguna kembali dari halaman lain) memakan waktu yang tidak pasti. Selama proses menunggu tersebut, ada kemungkinan widget asalnya sudah dihancurkan atau dihapus dari widget tree (misalnya karena aplikasi ditutup atau pengguna menekan tombol back dengan cepat). if (!mounted) mengecek apakah widget tersebut masih aktif. Jika kita menggunakan context (seperti menampilkan SnackBar atau Dialog) pada widget yang sudah mati (unmounted), aplikasi akan mengalami crash atau error fatal.